FINAL YEAR PROJECT · DEFENSE DOSSIER

CropSense AI

Crop Disease Detection System

A leaf photo goes in; a ranked disease hypothesis, a confidence band, a Grad-CAM heatmap and bilingual crop-care guidance come out — across three clients, one FastAPI service and one EfficientNetB2 model. This document is the complete walkthrough of how the system is built, why each decision was made, and where it is honestly weak.

Classes	52 — 51 disease/healthy + 1 Unknown catch-all
Crops covered	10 — Apple, Banana, Citrus, Cucumber, Grape, Maize, Mango, Potato, Rice, Tomato
Lab training images	38,348 across 51 classes
Real field images	1,592 across 51 classes
Backbone	EfficientNetB2 (ImageNet-pretrained), 224 × 224 × 3
Clients	React web app · React Native (Expo) mobile app · direct REST API
Stack	FastAPI · TensorFlow 2.21 · PostgreSQL 16 · Docker Compose
Source	Repository branch main, commit e70df98

Every figure, threshold, hyperparameter and file path in this document was read from the source code rather than reconstructed from the README. Where the README and the code disagree, the code is reported.

Contents

Right-click here and choose “Update Field” to build the table of contents.

1. What the system does

A farmer photographs a diseased leaf. Within a few seconds the system answers three questions: **what is probably wrong**, **how sure is it**, and **what should I do about it** — in English or Nepali, with a visual explanation of where the model was looking.

The framing throughout the product is deliberate. Every result is labelled a **possible match**, never a diagnosis. Confidence is surfaced as a band (High / Moderate / Uncertain) rather than a bare percentage, and every treatment recommendation carries a disclaimer telling the user to confirm dose, pre-harvest interval and local registration with an agrovet before spraying. This is a decision-support tool, not an oracle.

1.1 Scope

The model covers **10 crops** and **51 disease-or-healthy classes**, plus one deliberate 52nd class — `Unknown___Unknown` — that exists to absorb leaves the model was never trained on.

Crop	Classes	Conditions covered
Apple	4	Black Rot, Cedar Rust, Scab, Healthy
Banana	7	Black Sigatoka, Yellow Sigatoka, Bract Mosaic Virus, Insect Pest, Moko, Panama, Healthy
Citrus	3	Canker, Greening, Healthy
Cucumber	3	Downy Mildew, Powdery Mildew, Healthy
Grape	4	Black Measles, Black Rot, Leaf Blight, Healthy
Maize	3	Common Rust, Northern Leaf Blight, Healthy
Mango	8	Anthracnose, Bacterial Canker, Cutting Weevil, Die Back, Gall Midge, Powdery Mildew, Sooty Mould, Healthy
Potato	3	Early Blight, Late Blight, Healthy
Rice	6	Bacterial Leaf Blight, Brown Spot, Leaf Blast, Leaf Scald, Leaf Smut, Narrow Brown Spot
Tomato	10	Bacterial Spot, Early Blight, Late Blight, Leaf Mold, Mosaic Virus, Septoria Leaf Spot, Spider Mites, Target Spot, Yellow Leaf Curl Virus, Healthy
— (catch-all)	1	Unknown___Unknown

Class distribution by crop, from backend/class_names.json. Rice is the only crop with no Healthy class.

Labels use a strict `Crop__Disease` convention. The backend derives the supported-crop list by splitting on __ at startup, so the user-facing message “this crop is not covered” can never drift out of sync with what the model was actually trained on.

2. System architecture

Three clients, one service. The web app, the mobile app and any direct API consumer all hit the same `POST /predict` endpoint with the same multipart payload and get back the same JSON. There is no duplicated inference logic anywhere — a change to the confidence thresholds changes all three clients at once.

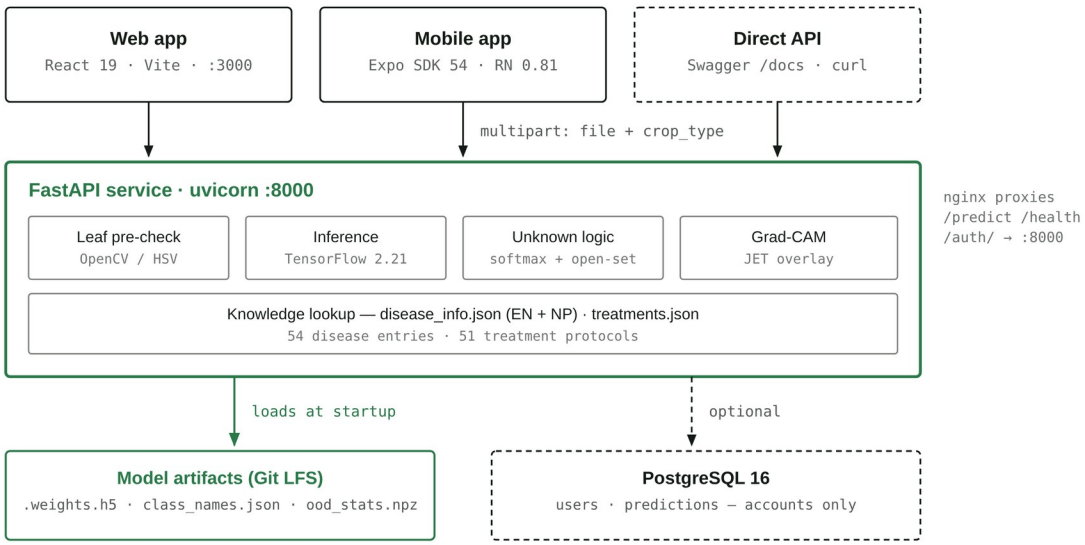

Figure 1 — System architecture. Solid arrows are the required path; dashed arrows are optional. The database sits on a dashed edge deliberately: init_db() failure is caught and logged at startup, so a missing Postgres degrades the app to "no accounts" rather than taking prediction down.

2.1 Why the database is optional

This is the single most defensible architecture decision in the project. The core value of the system — diagnose a leaf — has no dependency on user identity. So `/predict` takes no auth, touches no database, and the startup path wraps `init_db()` in a try/except that logs a warning and continues. Prediction history is instead stored **client-side** in `localStorage`. The result is that the whole app is usable with a single container and no persistence layer at all, and accounts are a genuinely additive feature rather than a gate.

2.2 Repository layout

Path	Contains
`backend/`	main.py (814 lines — model load, /predict, Grad-CAM, auth routes), auth.py, db.py, the JSON knowledge base, and the model weight folders
`frontend/`	React 19 SPA — 12 routes, 2 contexts, a normalization layer, CSS-Modules design system, Vitest suite
`mobile/`	Expo / React Native client — camera capture, crop chips, EN/NP toggle, shared result view
`scripts/`	Two Colab training notebooks (Kaggle and Google Drive variants) plus the field-data acquisition and cleaning pipeline
`database/`	schema.sql — mirrors what init_db() creates
`field_data/`	1,592 curated real-world field photos across 51 class folders
`docker-compose.yml`	Four services: db, backend, frontend, adminer

3. The data

Three distinct data sources feed the model, and they exist for three different reasons. Understanding this split is the key to defending the whole training design.

Source	Images	Character	Job it does
Lab set (Dataset.zip)	38,348	Clean, single leaf, plain background, controlled light	Teaches the visual signature of each disease
Field set (field_data/)	1,592	Real phone/camera photos: clutter, shadow, multiple leaves	Closes the lab→field domain gap
Unknown set	~1,000	Real leaves of untrained crops + synthetic filler	Teaches the model to decline

3.1 The imbalance problem

The lab set is severely imbalanced. `Maize__Common_rust` has 2,384 images; `Banana__Yellow_Sigatoka` has 23. That is a **104 : 1** ratio between the largest and smallest class. Left alone, a network trained on this will happily reach high overall accuracy by never predicting the rare classes at all — which is exactly why the project reports and checkpoints on **macro-F1** rather than accuracy.

Largest classes	n	Smallest classes	n
Maize Common Rust	2,384	Banana Yellow Sigatoka	23
Maize Northern Leaf Blight	2,384	Rice Leaf Smut	40
Grape Black Measles	1,920	Banana Panama	41
Grape Black Rot	1,888	Banana Bract Mosaic Virus	50
Maize Healthy	1,859	Banana Moko	55

The extremes of the lab set. The five largest classes hold more images than the twenty smallest combined.

3.2 How the field data was acquired

The field set was built rather than downloaded from an existing benchmark, and the acquisition pipeline is five scripts in `scripts/`. Being candid about the method matters more than dressing it up:

1. **download_field_images.py** — crawls Bing Images via `icrawler` with one hand-written query per class (“corn northern leaf blight field”, “rice bacterial leaf blight field”), targeting 50 raw images per class at a minimum size of 200 px.
2. **clean_field_images.py** — removes only *objective* junk with no label judgment: corrupt files, anything under 224 px on the short side, and near-duplicates inside a folder via 8×8 average-hash with a Hamming distance ≤ 4. Every deletion is logged to `clean_field_report.txt`.
3. **delete_collisions.py** — if the same image appears in two different class folders, at least one label is wrong, so *every* copy is deleted. On confusable disease pairs a wrong label costs more than a missing image.
4. **build_montages.py** + **apply_visual_deletions.py** — renders each class folder as a numbered contact sheet with a sidecar JSON index, so a human reviewer can say “delete tiles 3, 7, 12” and have that map back to real filenames. This is the manual label-verification step.

5. **redownload_thin.py** — re-fills classes that fell below target after cleaning, using multiple queries per class including *Latin pathogen names* (“apple frogeye leaf spot black rot Botryosphaeria”) to steer away from visual lookalikes.

OWN THIS IN THE DEFENSE

Web-scraped field data carries label noise that a curated benchmark does not. The pipeline mitigates it (dedup, collision deletion, visual review, pathogen-specific requery) but does not eliminate it, and there is no agronomist sign-off on the labels. Because the field test slice is drawn from the same scrape as the field training slice, any residual mislabelling is *correlated* across train and test — which would inflate the field score. Say this before the panel says it.

3.3 The Unknown class

Sections 3 and 4 of the training notebook build `Unknown___Unknown` up to `TARGET_UNKNOWN_TOTAL = 1000`, with synthetic content capped at `SYNTH_MAX_SHARE = 0.35`. The composition is deliberate and the notebook comments say why:

- **Real leaves of untrained crops** (guava, papaya, coffee…) do the actual work. A guava leaf looks like *a leaf*, so it lands near the healthy classes in feature space — only a real non-target leaf teaches the boundary.
- **Gaussian noise and crude leaf-like textures** only top the class up. They cover degenerate inputs — blur, sensor failure, a photo of nothing — and are explicitly *not* the mechanism that rejects an untrained crop.

4. Training methodology

The model is EfficientNetB2 with ImageNet-pretrained weights, fine-tuned in two phases. Every layer name in the notebook matches the architecture rebuilt in `backend/main.py`, which is what lets the backend load a bare `.weights.h5` file rather than a full serialized model.

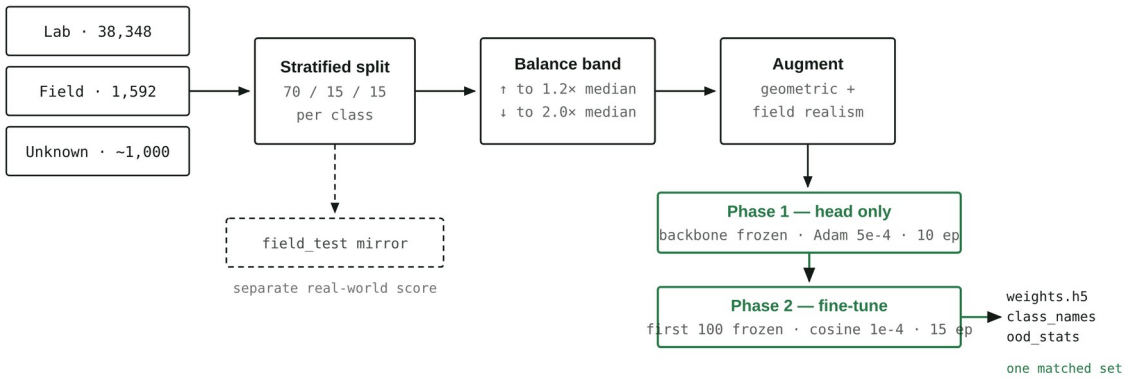

Figure 2 — Training pipeline. The three outputs on the right must always travel together. Centroids in ood_stats.npz live in the penultimate feature space of one specific checkpoint and are meaningless against any other set of weights.

4.1 Architecture

```
EfficientNetB2(include_top=False, weights='imagenet', input_shape=(224,224,3))
  -> GlobalAveragePooling2D()
  -> Dropout(0.5)
  -> Dense(512, relu,  name='dense_hidden', kernel_regularizer=l2(1e-4))
  -> Dropout(0.5)
  -> Dense(52, softmax, name='dense_output', dtype='float32',
                        kernel_regularizer=l2(1e-4))
```

Two details worth calling out. `dense_hidden` is named because the open-set rejection mechanism needs to tap that exact layer at inference time. And `dense_output` is pinned to `float32` because the run uses mixed-precision `float16` compute — softmax in fp16 is numerically unstable and would produce degenerate probabilities.

4.2 Hyperparameters

Setting	Phase 1	Phase 2	Rationale
Trainable	Head only	All but first 100	Early layers hold generic edge/texture filters worth keeping
Learning rate	5e-4 (Adam)	1e-4 → cosine, α=0.01	A high LR on an unfrozen pretrained backbone destroys the features
Epochs	10	15	Both cut short by EarlyStopping in practice
Batch size	64	64	Halves step count and keeps a T4 GPU fed
Loss	Categorical CE, label smoothing 0.1	same	Stops the network becoming over-confident — directly serves the confidence thresholds
Regularization	Dropout 0.5 ×2, L2	same	Small classes overfit fast

	1e-4		
Checkpoint monitor	val_macro_f1 (max)	same	Accuracy would let the saved model win by ignoring rare classes
EarlyStopping	val_loss, patience 7	same, restore best	—
Precision	mixed_float16	same	~1.5–2× throughput on tensor cores

From section 5 of scripts/train_colab_google.ipynb. Seeds are fixed at 42 for random, numpy and tensorflow.

4.3 The three-layer imbalance defence

The project does not rely on a single fix. It stacks three, in order, and each is deliberately gentle so they do not compound into over-correction:

6. **Undersample the majorities** down to 2.0× the median class count, and **oversample the minorities** up to 1.2× median (capped at 5× the real images, because repeating the same photo more than that just teaches memorization). This caps the ratio at roughly 2 : 1.
7. **Protected prefixes.** Undersampling deletes files, so the code refuses to delete anything prefixed `field_` (real field photos) or `leaf_` (real non-target leaves) — the two scarcest and most valuable kinds of image in the run. Synthetic filler is dropped first, then ordinary lab images.
8. **Sqrt-damped class weights:** `w_i = sqrt(mean_count / n_i)`. Because the counts are already inside a narrow band, these land near 1.0–1.5. The notebook comment records that the earlier `max_count / n` scheme (weights 1–6) applied *on top of* resampling double-corrected and destabilised training.

4.4 Augmentation, and why it is unusual

Standard geometric augmentation is present — rotation ±40°, width/height shift 0.25, shear 0.2, zoom 0.3, brightness 0.6–1.4, channel shift 30, horizontal flip, reflect fill. On top of that sits a custom `field_augment` function that exists specifically to attack the lab→field gap by making clean lab photos look like phone photos:

Transform	p	Parameters	Real-world artefact it simulates
Gaussian blur	0.35	kernel 3 or 5	Out-of-focus or motion-blurred phone shot
Additive noise	0.30	$\sigma \in [4, 14]$	Small sensor noise in poor light
Directional shadow	0.30	linear ramp, strength 0.5–0.85	Sun angle, the photographer’s own shadow

Vertical flip is deliberately disabled: leaves have a consistent tip-and-petiole orientation and an upside-down leaf is not a realistic input.

4.5 Practical engineering in the notebook

Three details worth mentioning if asked how the run was actually made to finish on free hardware:

- **Resume-safe checkpointing.** Phase-2 weights and an epoch counter are written to Google Drive after every epoch. If Colab disconnects mid-run, re-running all cells picks up from the last checkpoint instead of restarting.

- **Pre-resize on disk.** Every image is resized to 224×224 once, before training. Decoding a 4000×3000 phone photo costs more CPU per epoch than the forward pass costs GPU — and it would otherwise be paid every epoch.
- **Threaded loading.** Keras defaults to one loader thread; the run sets 8 workers with a 32-batch queue, using threads rather than processes because forked workers would inherit the same NumPy RNG state and repeat identical augmentations.

5. The inference pipeline

This is the part of the system to know cold. A request to `/predict` passes through a gauntlet with three separate exit points before it can produce a normal disease result.

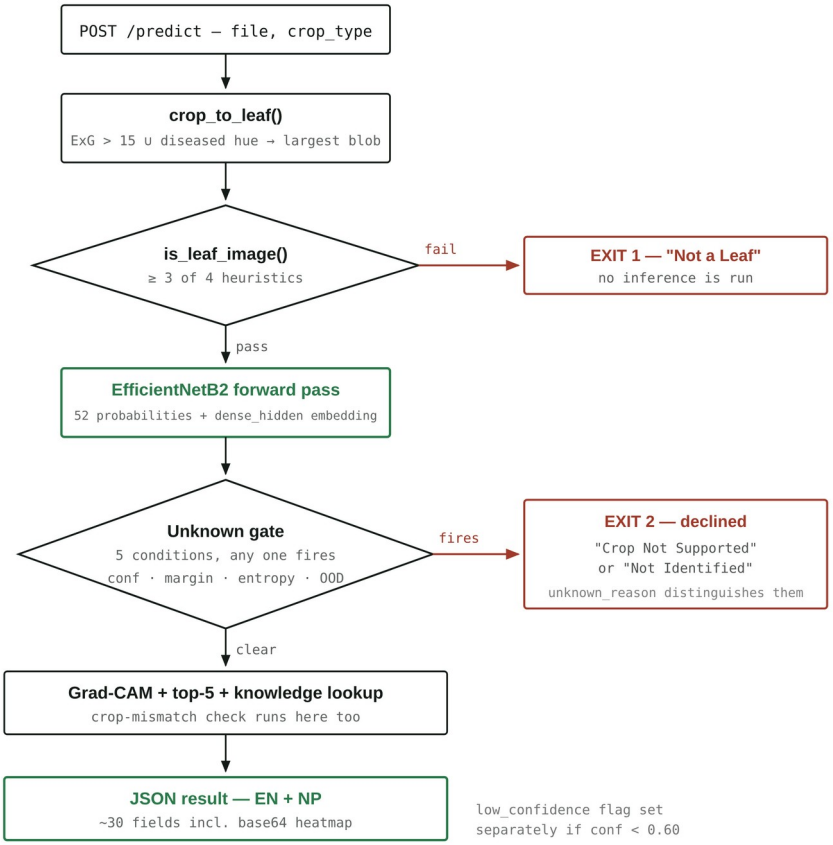

Figure 3 — The /predict request path. Two red exits, one green path. Both exits return a well-formed result object with the same schema, so the client never has to branch on HTTP status to know the model declined.

5.1 Step 1 — Leaf isolation (crop_to_leaf)

The model was trained mostly on centred lab leaves, so a busy field photo of soil, sky and neighbouring plants is out of distribution before the classifier even sees it. This step builds a foreground mask and crops to it:

- **Green tissue:** Excess-Green index `2G − R − B > 15`.
- **Diseased tissue:** HSV hue 15–45 with S > 110 and V > 90 — a deliberately *narrow* band, so dull brown soil (which shares that hue) is not swept in as foreground.
- Morphological open then close with a 7×7 ellipse, take the largest contour, pad its bounding box by 10%.

The safety rails matter more than the mask. If the blob covers less than 5% of the frame the mask is untrustworthy; if it covers more than 90% the leaf already fills the frame (a lab photo); if the resulting crop is thinner than 15% of either dimension it is

absurd. In all of those cases — and in any exception — the function returns the **original image unchanged**. It can only help, never mangle.

5.2 Step 2 — Leaf pre-check (is_leaf_image)

Four cheap independent heuristics; the image must pass at least three:

Test	Threshold	Catches
Leaf-colour ratio (green HSV 35–85 ∪ brown 10–35)	> 0.05	Photos with no vegetation at all
Greyscale standard deviation	> 20	Blank walls, flat textureless surfaces
Canny edge density	> 0.008	Out-of-focus or featureless frames
Aspect ratio (long ÷ short side)	> 1.1	Weak signal — most leaf photos are not square

Failing this returns a complete `"Not a Leaf"` result with `not_leaf: true` and actionable advice, without ever running the model. Note that the 3-of-4 rule is what makes it usable: the aspect-ratio test is genuinely weak, and a perfectly square crop of a real leaf still passes on the other three.

5.3 Step 3 — Preprocessing and forward pass

Resize to 224×224, then `efficientnet.preprocess_input` on raw 0–255 pixels — no manual `/255` rescaling, because EfficientNet's own preprocessing expects unscaled input. Getting this wrong is the classic silent accuracy killer, and the same function is used identically in the notebook, in `main.py`, and in the standalone tester `predict_test.py`.

When open-set statistics are loaded, a single dual-output model returns the class probabilities *and* the `dense_hidden` embedding from the same forward pass, so the rejection check costs no extra inference.

5.4 Step 4 — Post-processing

Beyond the Unknown gate (section 6), two other flags shape the result: **crop mismatch**, raised when the predicted crop differs from the crop the user selected, and **low confidence**, raised below `LOW_CONFIDENCE_THRESHOLD = 0.60`. These are independent of Unknown — a confident, in-distribution Tomato prediction on a photo the user tagged as Potato is neither unknown nor low-confidence, but is still worth a warning.

6. Teaching the model to say “I don’t know”

A 52-way softmax is closed-set: the probabilities must sum to 1, so a guava leaf — a crop the model has never seen — is forced onto some trained class, often with high confidence. Confidence thresholds alone cannot catch this. This is the most technically interesting problem in the project and the one most likely to be probed in a defense.

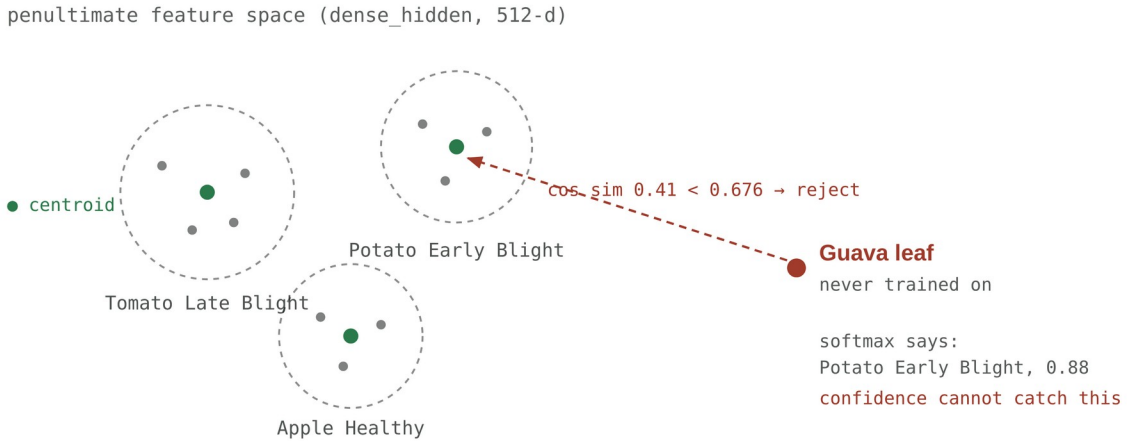

Figure 4 — Why softmax cannot reject an untrained crop. Softmax measures which trained class is relatively most likely; cosine distance to the class centroids measures whether the image resembles any trained class at all. The threshold shown, 0.6764, is the real calibrated value stored in final_field_Model/ood_stats.npz.

6.1 The five conditions

An input is declared unknown if *any one* of these fires. They are ordered here from cheapest to most sophisticated:

#	Condition	What it catches
1	`raw_class == "Unknown___Unknown"`	The model itself picked the catch-all class — including *confidently*, which the statistical tests below would miss entirely
2	`confidence < 0.30`	Flat-out uncertainty
3	`confidence < 0.50 and top1−top2 margin < 5 pp`	A near-tie between two classes — a coin flip dressed as a prediction
4	`normalized entropy > 0.90`	Probability mass smeared across many classes, even if one nominally leads
5	`max cosine similarity to any centroid < threshold`	An untrained crop the model is *confident* about — the case none of the above can reach

Entropy is normalized by `log(n_classes)` so it sits on [0, 1] regardless of how many classes the model has — the threshold survives adding a class.

6.2 How the open-set threshold is calibrated

9. Embed the entire training set through `dense_hidden` and L2-normalize.

10. Compute one centroid per *trained* class — the Unknown class is excluded, because it is a grab-bag with no coherent centre — giving 51 centroids of 512 dimensions.
11. Score a new image as its **highest cosine similarity to any centroid**.
12. Set the threshold at the 5th percentile of the scores of *known* validation leaves, so `TARGET_KNOWN_ACCEPT = 0.95` — the mechanism is tuned to wrongly reject at most 5% of genuine known crops. That is the explicit, quantified price paid for rejection.

6.3 Two kinds of “unknown” — and why the distinction is in the UI

Two very different situations reach this gate, and a user needs to know which one they hit. The backend sets `unknown_reason` accordingly:

- **unsupported_crop** → displayed as **“Crop Not Supported”**. Fires when the feature-distance rule triggers, or when the model confidently (≥ 0.50) picks the Unknown class. The message names the crops that *are* covered, generated from `class_names` so it is always accurate.
- **uncertain** → displayed as **“Not Identified”**. Message: *“Take a closer, sharper photo of a single leaf in even daylight and try again.”*

Telling a farmer “Unknown” for both would be a usability failure: one means *take a better photo*, the other means *this tool cannot help you at all*.

MUST KNOW — CURRENT DEPLOYMENT STATE

Open-set rejection is **currently inactive**. The backend defaults to `MODEL_PATH=last_final_model`, and that folder ships its statistics file as `odd_stats.npz` — a typo of `ood_stats.npz`, which is the exact filename `main.py` looks for. The file’s contents are also the wrong schema (four scalars: mean, std, min, threshold) rather than the expected 51×512 centroid array. So condition 5 never fires today; conditions 1–4 still do.

The previous checkpoint, `final_field_Model/`, has a valid `ood_stats.npz` (51 centroids × 512 dims, threshold 0.6764). Centroids are only valid against the weights they were computed from, so the fix is to re-export from the same run — not to copy the old file across. See section 14.

7. Explainability — Grad-CAM

Every successful prediction ships a heatmap showing which regions of the image drove the classification. This is what turns the system from a black box into something a user can sanity-check: if the model says Late Blight but the heat sits on a background twig, the user can see the result is untrustworthy without understanding a single thing about neural networks.

7.1 How it is computed

13. Walk the model in reverse to find the last layer whose output has rank 4 (a spatial feature map) inside the backbone.
14. Build a sub-model returning that layer's activations plus the backbone output; under a `GradientTape`, push the backbone output through the remaining head layers to recover the class logit.
15. Take gradients of that logit with respect to the conv activations, global-average-pool them into per-channel weights, and take the weighted sum of the feature maps.
16. ReLU, normalize to [0, 1], resize to the source image, apply `COLORMAP_JET`, and blend at 60% original / 40% heatmap.

7.2 Two bugs worth mentioning as engineering evidence

Both are documented in code comments and both are the kind of failure that is invisible rather than loud — good material if asked about debugging:

- **Keras 3 removed `Layer.output_shape`.** The original layer-walk raised `AttributeError` on every layer, so the search silently returned the base-model wrapper instead of a conv layer and Grad-CAM was disabled entirely — with no error surfaced. The fix (`_output_rank`) tries `layer.output.shape` first and falls back to the Keras 2 spelling.
- **`np.maximum` on a tensor already returns an ndarray**, so the following `.numpy()` call raised `AttributeError` — which the surrounding `except` swallowed, again silently disabling the heatmap.

HONESTY IN THE UI

The result page states, verbatim: *"Highlighted areas show which parts of the image influenced the model most. This visual explanation does not confirm the diagnosis."* Grad-CAM is an attribution method, not a validation method — a model can be confidently wrong and still produce a plausible-looking heatmap. Saying so out loud is the correct engineering position.

8. API surface

Method	Path	Auth	Purpose
GET	/	—	Status message
GET	/health	—	Liveness: model_loaded, model_backbone, number_of_classes
POST	/predict	—	The core endpoint. Multipart: file (required),

			crop_type (optional)
POST	/auth/signup	—	Create account — username, email, password
POST	/auth/login	—	Returns a JWT bearer token
GET	/auth/me	Bearer	Current user profile
GET	/auth/history	Bearer	Server-side prediction history
DELETE	/auth/history/{id}	Bearer	Delete one entry — ownership checked, 403 otherwise

8.1 The /predict response

Roughly 30 fields. The design principle: the client should never have to recompute anything the backend already knows, and every human-readable string ships in both languages.

Group	Fields
Identity	`disease, disease_np, raw_class, crop_type`
Confidence	`confidence, low_confidence, entropy, ood_score`
Gates	`is_unknown, not_leaf, crop_mismatch, unfamiliar_crop, unknown_reason, supported_crops`
Ranking	`top_5_predictions[], raw_probabilities[] (all 52)`
Guidance	`description, cause, symptoms, treatment, prevention — each with a _np twin`
Treatment protocol	`treatments[], treatment_disclaimer, treatment_disclaimer_np`
Explainability	`gradcam_image (base64 data URI)`
Copy	`message, disclaimer`

A small but real engineering detail: `_to_native()` walks the whole result recursively converting NumPy scalars to Python types. Without it FastAPI's JSON encoder throws on `np.float32` and `np.bool_`, and the failure appears as an opaque 500 rather than anything pointing at the model.

8.2 Structured treatment data

`treatments.json` holds 51 protocols with agronomically meaningful fields rather than free text — `active` (active ingredient), `kind` (chemical / cultural / biological), `band`, `formulation`, `dose`, `phi_days` (pre-harvest interval), `interval_days` (re-spray interval), and bilingual notes. The pre-harvest interval in particular is a food-safety figure, not a nicety.

Every treatment recommendation carries this disclaimer, in the user's chosen language:

DISCLAIMER — ENGLISH	अस्वीकरण — नेपाली
Always read and follow the product label. Doses are general guidance — confirm the rate, the pre-harvest interval and local registration with your agrovet or agriculture office before spraying. Wear gloves, a mask and long sleeves.	सधैं औषधिको लेबल पढेर पालना गर्नुहोस्। यहाँ दिइएको मात्रा सामान्य निर्देशन मात्र हो — छर्नु अघि मात्रा, बाली टिप्नु अघिको प्रतीक्षा अवधि र स्थानीय दर्ता आफ्नो कृषि पसल वा कृषि कार्यालयसँग पक्का गर्नुहोस्। पन्जा, मास्क र लामो बाहुला लगाउनुहोस्।

9. The web application

React 19 with Vite 8, React Router 7, Axios, and CSS Modules over a design-token file. Twelve routes, two React contexts, and — importantly — no component anywhere calls Axios directly or hardcodes a host.

Route	Page	Purpose
`/`	Home	Intro and the three-step workflow
`/diagnose`	Diagnose	Crop selector → photo → analyze, with all the validation states
`/result`	Result	Possible match, confidence band, Grad-CAM comparison, guidance, alternatives, feedback
`/history` · `/history/:id`	History	Local-first list with crop/status/sort/search filters, plus detail view
`/library` · `/library/:id`	Disease Library	Browsable reference for all 51 conditions
`/login` · `/register` · `/profile`	Accounts	Optional; only /profile is gated by RequireAuth
`/about`	About	What the tool does and how to use it responsibly

9.1 Where the confidence bands are decided

`src/lib/normalize.js` is the single source of truth for turning a raw response into what the UI renders. It is a pure function, which is exactly why it is the most-tested file in the project:

```
if (is_unknown || not_leaf || crop_mismatch || low_confidence)  ->  UNCERTAIN
else if (confidence >= 80)                                      ->  HIGH
else if (confidence >= 60)                                      ->  MODERATE
else                                                            ->  UNCERTAIN
```

Note the ordering: *any* backend doubt flag forces Uncertain regardless of the number. An 85%-confident prediction on a photo the model thinks is the wrong crop is not a High-confidence result, and the UI refuses to present it as one.

9.2 Client engineering worth pointing at

- **Duplicate-submit guard** — a `useRef` flag, not just the disabled state, because React state updates are asynchronous and a fast double-tap can slip through.
- **Typed error handling** — `classifyError()` maps failures into timeout / network / server / client, each with copy that says what to do next rather than surfacing a status code.
- **Input limits** — 12 MB upload cap and a 60-second request timeout, both enforced client-side before the user waits.
- **“Any crop — I’m not sure”** — a sentinel that sends no `crop_type`, disabling only the mismatch warning while leaving prediction untouched. It is honest about what it changes.
- **Two thumbnail sizes** — 160 px for the history record, 720 px for the result view, both generated on-canvas so the result survives a page refresh without bloating `localStorage`.

- **Storage-quota recovery** — if `localStorage` rejects a write, history is trimmed to half of `MAX_ENTRIES` and retried once rather than throwing.

9.3 Accessibility

Not an afterthought, and easy to demonstrate live: semantic landmarks, a skip link, keyboard-operable crop selection via a real `radiogroup`, `aria-live` status announcements during analysis, `role="alert"` on errors, meaningful alt text on both the photo and the heatmap, visible focus rings, `prefers-reduced-motion` support, 44 px touch targets, and status communicated by text as well as colour — which matters because the palette is red/amber/green and roughly 8% of men have some form of colour-vision deficiency.

10. The mobile application

React Native on Expo SDK 54 (RN 0.81.5, React 19.1). A deliberately thin client: it calls the same `/predict` endpoint and holds no inference logic of its own.

- **Camera or gallery** via `expo-image-picker`, quality 0.8, with runtime permission requests for each path.
- **Crop chips** mirroring the web selector, and the same duplicate-submit ref guard.
- **EN / NP toggle** in the header. `normalize.js` picks the `*_np` field when Nepali is active — the language switch is one state variable, and re-rendering the existing result requires no new network call.
- **40-second timeout**, shorter than the web app's 60 s, on the assumption of a mobile network and a user who will not wait.

10.1 Networking — the part that actually causes demo failures

The backend address is compiled in from `config.js`, overridable with `EXPO_PUBLIC_API_URL`. The committed default is a **Tailscale** tailnet address, which lets the phone reach a laptop-hosted backend from any network — mobile data included — provided the phone has Tailscale installed and signed in. For plain same-Wi-Fi testing you override with the laptop's LAN IP:

```
REACT_NATIVE_PACKAGER_HOSTNAME=192.168.77.106 \
EXPO_PUBLIC_API_URL=http://192.168.77.106:8000 \
npx expo start

# and the backend must bind all interfaces, not just loopback:
uvicorn main:app --host 0.0.0.0 --port 8000
```

DEMO-DAY CHECK

Before presenting, open `http://<backend-ip>:8000/health` in the **phone's own browser**. If that returns JSON, the app will work; if it does not, no amount of app debugging will help. Ship an APK built with `npx eas-cli build -p android --profile preview` as a fallback so the demo does not depend on Metro and a QR code.

11. Data storage and accounts

11.1 Local-first history

Every check is written to `localStorage` under `cropsense.history`, capped at 100 entries, with a 160 px thumbnail and the full text guidance but *not* the base64 Grad-CAM or the raw payload — those would blow the storage quota within a handful of records. History therefore works with no login, no database and no network.

11.2 PostgreSQL schema

Table	Columns
`users`	id serial PK · username unique · email unique · password (bcrypt hash) · created_at
`predictions`	id uuid PK (gen_random_uuid()) · username FK · timestamp · disease · disease_np · confidence · crop_type · is_unknown · not_leaf · message · cause/symptoms/treatment/prevention (EN + NP) · top_5_predictions (JSON text) · gradcam_image · thumbnail

Indexed on `username` and on `timestamp DESC` — the two access patterns the history endpoint actually uses.

11.3 Authentication

bcrypt for password hashing, JWT HS256 for sessions with a 24-hour expiry (`ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24`). One detail worth knowing because it looks like a bug and is not: `auth.py` truncates the password to 72 bytes before hashing. That is bcrypt's hard algorithmic limit — passing more raises in modern bindings — so the truncation is explicit rather than accidental.

BE READY FOR THIS QUESTION

`db_save_prediction(...)` is **commented out** inside `/predict`, and so is the `Depends(get_current_user)` parameter. That is why the endpoint needs no auth and no database — but it also means `GET /auth/history` will return an empty list on the running system, because nothing writes to the predictions table. The endpoint, the schema and the write function all work; they are simply not wired in. Present this as a deliberate consequence of the local-first design, not as an oversight, and be ready to uncomment the two lines if asked to show server-side history.

12. Deployment

One command brings up the whole stack:

```
git lfs pull                  # model weights are LFS pointers until you do
docker compose up --build
```

Service	Port	Image / build
Web app	3000 → 80	node:20-alpine build → nginx:alpine serve
Backend API	8000	python:3.10-slim + libgl1 for OpenCV
PostgreSQL	5432	postgres:16-alpine, healthchecked, named volume pgdata

Adminer	8080	Database GUI for inspecting tables live

The backend waits on the database's `pg_isready` healthcheck rather than merely on container start — `depends_on: condition: service_healthy`. Nginx serves the SPA with a `try_files` fallback to `index.html` (so deep links like `/history/abc` resolve) and proxies `/predict`, `/health` and `/auth/` to `backend:8000` with a 300-second read timeout and a 50 MB body limit — a cold TensorFlow model plus a large phone photo can genuinely take a while on the first request.

12.1 Hosting realities

TensorFlow plus a ~100 MB model needs roughly 1.5–2 GB of RAM, so most 512 MB free tiers will OOM on import. The Dockerfile is built for this: it creates a UID-1000 user with a writable `$HOME` (which is what Hugging Face Spaces runs as, and what TensorFlow/Keras need for their caches), and its `CMD` is in shell form so `${PORT:-8000}` expands at runtime — Cloud Run injects `$PORT` and ignores `EXPOSE`. `backend/README.md` carries the Hugging Face Spaces YAML front matter (`sdk: docker`, `app_port: 8000`).

12.2 Version pinning

`requirements.txt` pins every dependency exactly, with a comment explaining why: the Grad-CAM layer walk handles Keras 3's removal of `Layer.output_shape`, and an unpinned TensorFlow upgrade on a rebuild would silently disable the heatmap again — the exact bug described in section 7. This is a good example to cite if asked about reproducibility.

13. Evidence and results

READ THIS BEFORE QUOTING ANY NUMBER

Both training notebooks in the repository have **zero saved cell outputs**. There is no stored classification report, no macro-F1 figure, and no field-only accuracy score for the currently deployed checkpoint. The only saved artifacts are two PNGs in `backend/Final_Model/` from a run dated 31 Aug 2026. Quote those as what they are, and do not attribute them to the deployed model.

13.1 What the saved training curves show

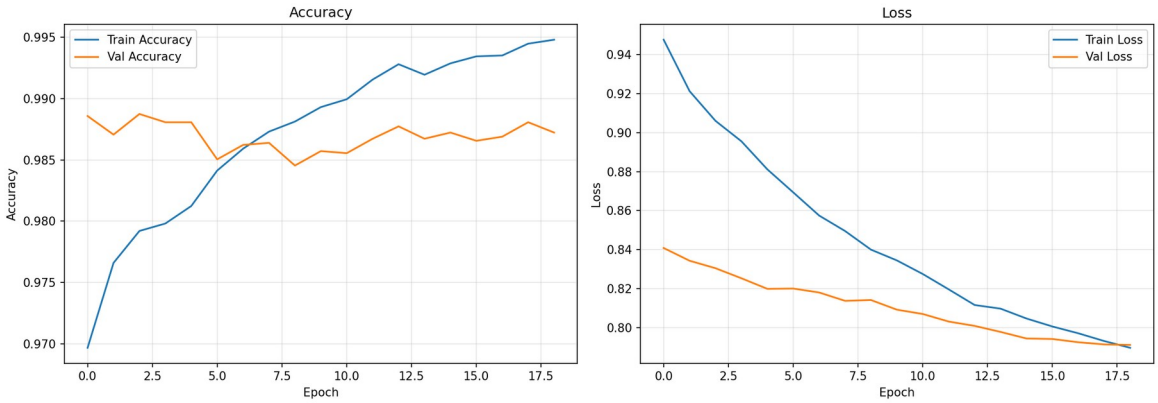

Figure 5 — Accuracy and loss across 19 epochs, from backend/Final_Model/training_history.png (run of 31 Aug 2026).

The plot shows:

- **Validation accuracy** flat in a band of roughly **0.985–0.989**, peaking near the start and again around epoch 17.
- **Training accuracy** climbing from 0.970 to about **0.995**, crossing above validation accuracy around epoch 6.
- **Both losses** falling and converging (train 0.95 → 0.79, validation 0.84 → 0.79).

Three things to be able to explain about that plot, because a panel will ask:

17. **Why validation accuracy starts *above* training accuracy.** Training metrics are measured with dropout active and augmentation applied; validation is measured clean. Early on, this makes the training numbers look worse than the model actually is. This is expected, not a bug.
18. **Why loss plateaus near 0.79 instead of approaching zero.** Label smoothing of 0.1 puts a floor under cross-entropy — the target is never a one-hot vector, so the loss cannot go to zero. Absolute loss values are not comparable to an unsmoothed run.
19. **The train/validation gap widening after epoch 6** is mild overfitting. Validation accuracy does not *degrade*, it plateaus — which is why EarlyStopping on `val_loss` with `restore_best_weights` and macro-F1 checkpointing are the right guards.

13.2 What the confusion matrix shows

Figure 6 — Test-set confusion matrix from the same run. Note it has 51 rows, not 52.

Strongly diagonal with very little off-diagonal mass — visually consistent with the ~98.7% validation accuracy. Two caveats to raise yourself:

- It has **51 rows, not 52** — there is no Unknown row, so this matrix predates the Unknown class being added.
- It is a **lab-set** test score. ~98–99% on a curated lab split is the well-known inflated figure that PlantVillage-style datasets produce, and it is not what a farmer will experience in a field. The number that matters is the field-only score, and that is exactly the number the repository does not currently have saved.

13.3 Evaluation the notebook is set up to produce

The instrumentation is written and ready — it simply needs a run with outputs preserved:

Section	Produces	Status
14	Overall test accuracy and loss	Ready
15	Per-class precision / recall / F1 + field-only accuracy and macro-F1 on the mirrored field_test split	Ready
15b	Unknown recall, false-unknown rate on known leaves, and untrained-crop rejection using the backend's exact decision rule	Needs HOLDOUT_UNTRAINED set — currently None
15c	Open-set cost/benefit: fraction of known leaves	Ready

	wrongly rejected vs. unknowns caught	
17 / 18	Confusion matrix and training curves	Ready

Section 15b deserves a specific mention: it re-implements `backend_decision()` to mirror `main.py`'s rule exactly, with a comment instructing that the two be kept in sync. That means the rejection number the notebook reports is the number users actually experience — evaluation measures the deployed behaviour, not an idealised version of it.

13.4 Software testing — verified

The frontend test suite runs green: **16 tests across 3 files, all passing** (Vitest 3.2.7 + React Testing Library, jsdom). Coverage targets the logic most worth protecting rather than chasing a percentage:

File	Tests	What it protects
`lib/normalize.test.js`	8	Response normalization and the High/Moderate/Uncertain derivation
`lib/history.test.js`	4	The local-first history store
`components/ResultView.test.jsx`	4	Responsible wording, the Grad-CAM note, the uncertainty warning, feedback controls

For the backend, `predict_test.py` is a standalone tester that rebuilds the same architecture, weight-load order, preprocessing and confidence threshold as the API with no web, auth or database dependencies — so a model can be verified against a folder of images without bringing up the stack.

13.5 What to generate before the defense, if there is time

20. Re-run the notebook end to end **and save the outputs**, so per-class F1 and the field-only score exist for the deployed checkpoint.
21. Set `HOLDOUT_UNTRAINED` to a folder of leaves from crops kept entirely out of training (guava, papaya, coffee) and record the rejection rate. This is the single number that proves the Unknown machinery works.
22. Re-export `ood_stats.npz` from that same run, correctly named, so open-set rejection is actually live in the demo.

14. Known gaps — state these before you are asked

A panel finds weaknesses either way. Naming them first converts each one from a hole in the work into evidence that you understand your own system.

[Blocking] Open-set rejection is off in the default deployment
The file is named `odd_stats.npz` where `main.py` looks for `ood_stats.npz`, and its contents are four scalars instead of the 51×512 centroid array. Fix by re-exporting from section 15c of the same training run — do not copy the older file, because centroids are only meaningful against the weights they were computed from.

[Blocking] No saved metrics for the deployed model
Both notebooks have zero cell outputs. There is no macro-F1, no per-class report, and no field-only accuracy for `last_final_model` — only PNGs from an earlier 51-class run. This is the highest-value thing to fix before the defense.

[Substantive] Untrained-crop rejection was never measured
`HOLDOUT_UNTRAINED = None`, so the rejection test in sections 15b and 15c has never run against a real held-out untrained crop. The mechanism is designed and implemented; its effectiveness is currently unquantified.

[Substantive] Field labels are web-scraped and expert-unverified
1,592 images from Bing search, cleaned by automated dedup and human visual review, but with no agronomist sign-off. Because train and test slices come from the same scrape, residual label noise is correlated and would inflate the field score.

[Substantive] Lab accuracy is not field accuracy
~98–99% on a curated lab split is the standard inflated PlantVillage-style figure. Field data, field-realism augmentation and leaf cropping all exist to narrow this gap, but the gap is real and must not be presented as real-world performance.

[Security] Development-grade auth configuration
`JWT_SECRET` falls back to a hardcoded default, and CORS is `allow_origins=["*"]`. Both are fine for a demo and both must be tightened before any real deployment. Being able to say exactly that is better than being caught by it.

[By design] Server-side history is not wired in
`db_save_prediction(...)` is commented out in `/predict`, so `/auth/history` returns empty. A consequence of the local-first architecture — two lines from working.

[Hygiene] Documentation drift and stale artifacts
The root `README.md` still describes `backend/best_model/` as the active model and does not mention leaf cropping, open-set rejection or `treatments.json`. The root `class_names.json` is a stale 24-class leftover (unused — the backend reads the copy inside the model folder). Five model folders are present with no manifest saying which is which.

[Limitation] Single-leaf, single-disease assumption
The classifier assigns one label to one image. Co-infections, nutrient deficiencies that mimic disease, and disease severity or staging are all outside the model's

vocabulary. Grad-CAM is attribution, not verification — a confidently wrong prediction still produces a plausible heatmap.

15. Defense Q&A

Q. Why EfficientNetB2 rather than ResNet, VGG, or a larger EfficientNet?

Accuracy per parameter. B2 hits ~9 M parameters against VGG16's 138 M, so it trains on a free Colab T4 and serves on a CPU-only free tier — real constraints for this project. Compound scaling means B2 balances depth, width and resolution together rather than scaling one axis. B3 was tried first (`scripts/train.py`, 300×300) and dropped: the larger input costs quadratic compute for a marginal gain on 224 px-native source images, and the deployment memory budget could not absorb it.

Q. You report 98%+ accuracy. Would this actually work in a farmer's field?

Not at that number, no — and the project is built around admitting it. That figure is a lab-set score on curated, plain-background images, and it is the well-documented inflated result that PlantVillage-style datasets produce. Three mechanisms exist specifically to narrow the gap: 1,592 real field photos merged into training, the `field_augment` pipeline that adds blur, sensor noise and directional shadow to lab images, and `crop_to_leaf` at inference which isolates the leaf from a cluttered scene before classification. The notebook maintains a separate `field_test` split to measure the residual gap — and that measurement is the number I would want to report, which is exactly the number currently missing.

Q. What happens if I photograph my hand? Or a crop you never trained on?

Three different defences, in order. A hand fails `is_leaf_image` — insufficient leaf-colour ratio, wrong edge profile — and returns "Not a Leaf" without running the model. An untrained crop passes that check because it genuinely is a leaf, so it hits the Unknown gate: a low-confidence, high-entropy, or near-tie prediction is declined statistically, and the open-set rule catches the harder case where the model is *confidently* wrong by measuring cosine distance to the trained-class centroids in feature space. And the training set contains a real Unknown class of non-target leaves, so the model can name that outcome directly. I should note that the open-set component is currently disabled by a filename mismatch in the deployed checkpoint — conditions 1 through 4 are active.

Q. Why can't softmax confidence alone reject an unknown crop?

Because softmax is closed-set — the outputs are constrained to sum to 1 across the trained classes. It answers "which of my 52 classes is relatively most likely", never "does this belong to any of them". A guava leaf shares the visual grammar of the trained leaves, so the network genuinely can be 88% confident it is Potato Early Blight. No confidence threshold reaches that case. Distance in feature space does, because it asks a different question: how far is this from anything I have seen. That is why the two rules are OR-ed rather than one replacing the other.

Q. Your dataset is severely imbalanced — 2,384 images in one class and 23 in another. How did you handle that?

Four layers, applied gently so they do not compound. Undersample majorities to 2.0× the median and oversample minorities to 1.2× median, which caps the ratio

at about 2:1. Protect real field photos and real non-target leaves from ever being the images deleted. Apply sqrt-damped class weights `sqrt(mean/n)` — the earlier `max/n` scheme gave weights of 1–6 which, stacked on top of resampling, double-corrected and destabilised training. And most importantly: checkpoint and report on **macro-F1**, not accuracy, so the saved model cannot win by favouring the crops with the most images.

Q. Why two training phases instead of just fine-tuning everything?

The classifier head starts with random weights. Unfreezing the pretrained backbone while random gradients flow back through it destroys the ImageNet features you paid for in the first place. Phase 1 freezes the backbone and trains only the head at 5e-4 until the head produces sensible gradients. Phase 2 then unfreezes — except the first 100 layers, which hold generic edge and texture filters worth keeping — and fine-tunes at 1e-4 on a cosine decay to 1e-6. Two orders of magnitude between the phase learning rates is the whole point.

Q. What does the Grad-CAM heatmap actually prove?

That the prediction was driven by the lesion rather than by the background — and nothing more than that. It is an attribution method: it shows where gradient signal for the chosen class concentrated in the last convolutional feature map. It does not verify that the class is correct; a confidently wrong model produces a plausible heatmap over a real lesion it has misidentified. The UI says this in as many words. Its practical value is catching the obvious failure: if the heat sits on a background twig or the photographer's thumb, the user knows to retake the photo without needing to understand the model.

Q. Why is there no login requirement?

Because requiring an account to diagnose a leaf would be a product failure for the intended user. The diagnosis is the value, and it has no dependency on identity — so `/predict` takes no auth and touches no database, `init_db()` failure is caught and logged rather than fatal, and history lives in browser `localStorage`. The whole system runs on a single container with no persistence layer. JWT auth, bcrypt and the Postgres schema are all implemented and working; accounts are genuinely additive rather than a gate.

Q. Where did the field images come from, and how do you know the labels are right?

Bing image search via `icrawler`, with a hand-written query per class, then a five-stage cleaning pipeline: remove corrupt and sub-224 px files, remove near-duplicates within a folder by 8×8 average hash at Hamming distance ≤ 4, delete *every* copy of any image appearing in two class folders (at least one label is wrong and a wrong label on a confusable pair costs more than a missing image), review each class as a numbered contact sheet and delete by tile index, then re-download thin classes using Latin pathogen names to avoid lookalikes. I want to be direct: this is not expert-verified data. Residual label noise exists, and because train and test come from the same scrape, that noise is correlated — which would inflate the field score. An agronomist review pass is the correct next step.

Q. Why does the model reject some real leaves as unknown?

Because that is the calibrated price of rejection, and it is a number rather than an accident. The open-set threshold is set at the 5th percentile of known-class

validation scores, so `TARGET_KNOWN_ACCEPT = 0.95` — up to 5% of genuine known-crop leaves are expected to be wrongly rejected. Raising the threshold catches more unknowns and rejects more real leaves; lowering it does the reverse. For an advisory agricultural tool, "I'm not sure, take another photo" is a far cheaper error than confidently naming the wrong disease and sending someone to buy the wrong pesticide.

Q. Why crop the leaf before classifying? Doesn't that risk destroying information?

It would, which is why every failure path returns the original image untouched. The model is trained mostly on centred lab leaves, so a field photo of soil, sky and neighbouring plants is out of distribution before the classifier sees it. `crop_to_leaf` builds a mask from Excess-Green plus a deliberately narrow diseased-tissue hue band — narrow so brown soil, which shares that hue, is not swept in — and crops to the largest blob. Then the rails: if the blob is under 5% of the frame the mask is untrustworthy, over 90% means it is already a lab photo, and a crop thinner than 15% of either dimension is rejected. Any of those, or any exception, and the original goes through unchanged. It can only help.

Q. Why is everything bilingual?

Because the intended user is a Nepali farmer, and an English-only agricultural tool is not usable by the people it is for. It is not a translation layer bolted on top — every content field in the knowledge base is stored as a matched pair (`symptoms` / `symptoms_np`), the API returns both in every response, and the clients pick per the language toggle. That means switching language re-renders an existing result with no new network call, and it means the safety disclaimer — the pesticide-label warning — is never the part that fails to translate.

Q. What would you do next, with more time?

In order of value. First, re-run training with outputs saved and set `HOLDOUT_UNTRAINED`, so per-class F1, the field-only score and the rejection rate all exist as measured numbers — the machinery is written, it just has not been run to completion with results preserved. Second, get an agronomist to verify the field labels. Third, replace the heuristic `crop_to_leaf` with a proper segmentation model, which would generalise far better than hand-tuned HSV thresholds. Fourth, add disease *severity* — a farmer needs to know how bad it is, not only what it is. Fifth, offline on-device inference via TFLite, because rural connectivity is exactly the constraint this tool runs into.

16. Live demo script

A sequence that shows the whole system in about five minutes and — importantly — demonstrates the failure handling, which is what separates this from a notebook with a good accuracy number.

#	Do	Say
1	Open :8000/health	Model loaded, backbone, 52 classes. Proves the service is live before anything else.
2	Diagnose a clear Tomato Late Blight leaf	The happy path: High confidence, Grad-CAM sitting on the lesion, bilingual guidance, structured treatment with pre-harvest interval.
3	Toggle EN → NP	Instant — the response already carries both languages, no second request.
4	Upload a photo of a hand or a wall	“Not a Leaf” — rejected before inference, with advice on what to photograph instead.
5	Upload a leaf of an untrained crop	“Crop Not Supported”, listing the ten crops that are covered. Explain the closed-set softmax problem here.
6	Select Potato, upload a Tomato leaf	Crop-mismatch warning — a prediction can be confident and still be flagged.
7	Upload a blurry or partly shaded leaf	Moderate or Uncertain band, plus “other possibilities” with their confidences.
8	Open History, then the phone	Local-first history with no login; same backend, same result, from the mobile client.
9	Open :8000/docs	Swagger. Shows the API is a real contract, not a UI-coupled endpoint.

16.1 Have these open in tabs before you start

- `frontend/src/lib/normalize.js` — the confidence-band logic, in case they ask where the bands come from.
- `backend/main.py` at the Unknown gate — the five conditions in one screen.
- Section 15c of the notebook — the centroid calibration.
- `backend/Final_Model/confusion_matrix.png` and `training_history.png`.

16.2 Prepared answers for the two questions that will land hardest

- **“What’s your macro-F1?”** — Do not guess a number. Say the metric is what the run checkpoints and early-stops on, that the notebook computes the full per-class report and a separate field-only score, and that the current checkpoint’s outputs were not preserved. Point at what the evaluation measures rather than inventing a figure. A panel forgives a missing number; it does not forgive a fabricated one.
- **“Show me it rejecting an unknown crop.”** — Test this before the defense with the actual deployed checkpoint. With the open-set rule inactive, rejection currently rests on conditions 1–4, so whether a given guava photo is declined depends on how the softmax lands. Know the answer for the specific image you plan to use.

Compiled from the CropSense repository at commit e70df98 (branch main). Every figure, threshold, hyperparameter and file path in this document was read from the source rather than reconstructed from the README — where the README and the code disagree, the code is reported. Frontend test results were verified by running the suite.